

Migrating from HTTP-Digest- Signature to JWT Authentication

Contents

Migrating from HTTP-Digest-Signature to JWT Authentication	2
Step 1: Add the JWT auth builder script.....	2
Step 2: Update `cartridge/scripts/secureacceptance/adapters/Flex.js`	3
Step 3: Update `cartridge/scripts/jobs/DMOrderStatusUpdate.js`	4
Step 4: Verify.....	5

Migrating from HTTP-Digest-Signature to JWT Authentication

For merchants who want to stay on their current cartridge version (assumed here to be 26.1.2) and apply the JWT authentication change directly to their existing code, instead of upgrading to a newer cartridge release.

Cybersource is deprecating HTTP-Digest-Signature messaging for REST calls. This guide replaces it with JWT authentication for the three affected REST calls:

- Flex Token Service (/microform/v2/sessions)
- Asymmetric Key Management (/flex/v2/public-keys)
- Conversion Detail Report (/reporting/v3/conversion-details)

Response Message-Level Encryption is a separate, optional feature and is not part of this guide.

No new credentials are required - the existing SA_Flex_KeyID, SA_Flex_SharedSecret, and SA_Flex_HostName site preferences are reused as-is. No metadata changes are required either.

Step 1: Add the JWT auth builder script

Create cartridge/scripts/mle/jwtAuthBuilder.js:

```
'use strict';

function base64UrlEncode(input) {
  var Encoding = require('dw/crypto/Encoding');
  var Bytes = require('dw/util/Bytes');
  var bytes = (typeof input === 'string') ? new Bytes(input, 'UTF-8') : input;
  return Encoding.toBase64URL(bytes);
}

function getDigestBase64(digestString) {
  var MessageDigest = require('dw/crypto/MessageDigest');
  var Bytes = require('dw/util/Bytes');
  var Encoding = require('dw/crypto/Encoding');
  var digester = new MessageDigest(MessageDigest.DIGEST_SHA_256);
  var digest = digester.digestBytes(new Bytes(digestString, 'UTF-8'));
  return Encoding.toBase64(digest);
}

function buildJWTAuthHeader(params) {
  var Mac = require('dw/crypto/Mac');
  var Encoding = require('dw/crypto/Encoding');
  var Bytes = require('dw/util/Bytes');
  var UUIDUtils = require('dw/util/UUIDUtils');

  var iat = Math.floor(new Date().getTime() / 1000);
```

```

    var exp = iat + 120;

    var header = {
      alg: 'HS256',
      kid: params.keyID,
      typ: 'JWT'
    };
    var payload = {
      iat: iat,
      exp: exp,
      iss: params.merchantId,
      jti: UUIDUtils.createUUID(),
      'v-c-jwt-version': '2',
      'v-c-merchant-id': params.merchantId,
      'request-host': params.requestHost,
      'request-method': params.requestMethod,
      'request-resource-path': params.requestResourcePath
    };
    if (params.digestString) {
      payload.digest = getDigestBase64(params.digestString);
      payload.digestAlgorithm = 'SHA-256';
    }

    var headerB64 = base64UrlEncode(JSON.stringify(header));
    var payloadB64 = base64UrlEncode(JSON.stringify(payload));
    var signingInput = headerB64 + '.' + payloadB64;

    var mac = new Mac(Mac.HMAC_SHA_256);
    var secretBytes = Encoding.fromBase64(params.sharedSecret);
    var signatureBytes = mac.digest(new Bytes(signingInput, 'UTF-8'), secretBytes);
    var signatureB64 = base64UrlEncode(signatureBytes);

    return signingInput + '.' + signatureB64;
  }

  function applyJWTAuth(signedHeaders, params) {
    var jwt = buildJWTAuthHeader(params);
    signedHeaders.put('Authorization', 'Bearer ' + jwt);
  }

  module.exports = {
    buildJWTAuthHeader: buildJWTAuthHeader,
    applyJWTAuth: applyJWTAuth
  };
};

```

Step 2: Update `cartridge/scripts/secureacceptance/adapters/Flex.js`

In `CreateFlexKey()`, replace the HTTP-Digest-Signature block:

```

// Before
signedHeaders.put('host', host);
signedHeaders.put('date', getTime());
signedHeaders.put('request-target', 'post /microform/v2/sessions?format=JWT');
signedHeaders.put('digest', getDigest(digestString));
signedHeaders.put('v-c-merchant-id', merchantId);
signature = generateSignature(signedHeaders, keyID, sharedSecret);
var headerString = '';
collections.forEach(signedHeaders.keySet(), function (key) {

```

```

        headerString = headerString + ' ' + key;
    });
    var signatureMap = new HashMap();
    signatureMap.put('keyid', keyID);
    signatureMap.put('algorithm', 'HmacSHA256');
    signatureMap.put('headers', headerString);
    signatureMap.put('signature', signature);
    var signaturefields = '';
    collections.forEach(signatureMap.keySet(), function (key) {
        signaturefields = signaturefields + key + '=' + signatureMap.get(key) + '"';
    });
    signaturefields = signaturefields.slice(0, signaturefields.length - 2);
    signedHeaders.put('signature', signaturefields);
    signedHeaders.remove('request-target');

```

```

// After
var jwtAuthBuilder = require('*/cartridge/scripts/mle/jwtAuthBuilder');
jwtAuthBuilder.applyJWTAuth(signedHeaders, {
    keyID: keyID,
    merchantId: merchantId,
    sharedSecret: sharedSecret,
    requestHost: host,
    requestMethod: 'post',
    requestResourcePath: '/microform/v2/sessions?format=JWT',
    digestString: digestString
});

```

In `getPublicKey()`, replace the equivalent HTTP-Digest-Signature block the same way:

```

// After
var jwtAuthBuilder = require('*/cartridge/scripts/mle/jwtAuthBuilder');
jwtAuthBuilder.applyJWTAuth(signedHeaders, {
    keyID: keyID,
    merchantId: merchantId,
    sharedSecret: sharedSecret,
    requestHost: host,
    requestMethod: 'get',
    requestResourcePath: '/flex/v2/public-keys/' + kid
});

```

Once both blocks are replaced, the `getTime()`, `getDigest()`, and `generateSignature()` functions in this file are no longer called and can be removed, along with their entries in `module.exports`.

Step 3: Update `cartridge/scripts/jobs/DMOrderStatusUpdate.js`

In `orderStatusUpdate()`, replace the equivalent HTTP-Digest-Signature block:

```

// After
var jwtAuthBuilder = require('*/cartridge/scripts/mle/jwtAuthBuilder');
jwtAuthBuilder.applyJWTAuth(signedHeaders, {
    keyID: keyID,
    merchantId: merchantId,
    sharedSecret: sharedSecret,
    requestHost: host,

```

```
        requestMethod: 'get',  
        requestResourcePath: '/reporting/v3/conversion-details?startTime=' + time.start +  
        '&endTime=' + time.end + '&organizationId=' + merchantId  
    });
```

The `getTime()` and `generateSignature()` functions in this file are then no longer called and can be removed.

Step 4: Verify

- Place a test order using Flex Microform.
- Confirm the response status is OK (not a 401).